# Certifying Minimum-Cost Phase Refinements for BFT Testing

**Abstract.** Sharing one admission bit across voting phases can remove a BFT test's agreement counterexample: additional evidence may repair a validator before it signs an indispensable conflicting vote. We turn this observation into a constructive design problem: which phase distinctions restore a witness at minimum cost? For an explicit independent-pivot certificate schema, we characterize weighted feasibility under arbitrary finite opportunity orders, unequal control costs, and allowed or required keys. A derived repair graph connects reusable contributors to jointly realizable event reversals. Component-wise enumeration and quota dynamic programming produce a legal model execution plus independently checked optimality or infeasibility certificates. A three-key family shows that the control value is not submodular and that immediate-gain-per-cost selection can be arbitrarily expensive. Lean mechanizes graph/event-order realization and the Bellman certificate principle. Two raw-event oracles validate 35,223 and 3,253 optimum certificates; structural cases reach 48 pivots with explicit resource limits. A guarded four-validator model supplies the disagreement witness. Separately, four normal-operation scenarios execute unchanged CometBFT code and validate 32 native signatures. These conformance checks support selected abstraction obligations; they do not reproduce the model's mutation or establish a full client refinement.

Best student paper eligibility: to be confirmed by the authors.

# Certifying Minimum-Cost Phase Refinements for BFT Testing

Anonymous author
Anonymous affiliation

## Abstract

Sharing one admission bit across voting phases can remove a BFT test's agreement counterexample: additional evidence may repair a validator before it signs an indispensable conflicting vote. We turn this observation into a constructive design problem: which phase distinctions restore a witness at minimum cost? For an explicit independent-pivot certificate schema, we characterize weighted feasibility under arbitrary finite opportunity orders, unequal control costs, and allowed or required keys. A derived repair graph connects reusable contributors to jointly realizable event reversals. Component-wise enumeration and quota dynamic programming produce a legal model execution plus independently checked optimality or infeasibility certificates. A three-key family shows that the control value is not submodular and that immediate-gain-per-cost selection can be arbitrarily expensive. Lean mechanizes graph/event-order realization and the Bellman certificate principle. Two raw-event oracles validate 35,223 and 3,253 optimum certificates; structural cases reach 48 pivots with explicit resource limits. A guarded four-validator model supplies the disagreement witness. Separately, four normal-operation scenarios execute unchanged CometBFT code and validate 32 native signatures. These conformance checks support selected abstraction obligations; they do not reproduce the model's mutation or establish a full client refinement.

**2012 ACM Subject Classification** Theory of computation → Distributed algorithms; Computer systems organization → Dependable and fault-tolerant systems and networks; Software and its engineering → Formal methods

**Keywords and phrases** Byzantine fault tolerance, consensus testing, scheduler synthesis, certifying algorithms, phase refinement, formal methods

**Digital Object Identifier** 10.4230/LIPIcs.OPODIS.2026.0

## 1 Introduction

Fault-injection schedulers determine which behaviors a test can observe. A scheduler may choose one communication partition for a protocol view, one directed admission bit per peer pair, or distinct decisions for different voting phases. Twins, randomized Byzantine testing, Netrix, and ByzzBench demonstrate the practical value of controlled execution [2, 13, 6, 1]. Their capabilities also make a semantic question unavoidable: when does aggregating admission decisions preserve a safety counterexample?

Consider an enabling prevote and a later precommit carrying repair evidence. They share a directed peer connection and a protocol round. A phase-sensitive policy can admit the former and suppress the latter. An admission bit fixed across both phases admits both or neither. Admitting more evidence may change a validator's state before it issues a vote needed by a conflicting quorum. Thus counting possible schedules does not establish preservation of the observations that matter to a safety test.

The critical boundary is **vote issuance**. If a repair message arrives after an indispensable conflicting precommit has been signed, it cannot remove that precommit from another validator's future certificate. Repair before the second *decision* can therefore be too late. We call the collection of second-vote opportunities a **repair frontier** and study which pivotal contributions can jointly survive across it.

The practical question is constructive: **which admission keys must distinguish**

© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0

**phases, and at what minimum cost?** Some keys may be unavailable to an adapter; others may already be required. A coarse policy can assign different initially ready validators to the two branches, while dependencies may prevent several individually delayable repairs from being delayed together. Quorum deficits, indivisible vote weights and the chosen keys therefore have to be considered jointly.

## 1.1 Contributions and scope

We give a certifying procedure that minimizes phase-refinement cost in an explicit independent-pivot certificate schema. Its starting point is an exact characterization: if there are $k$ unit-weight pivots, quorum deficits $d_0, d_1$, and maximum jointly reusable capacity $\alpha(H)$ in a derived repair graph, a coarse unsafe execution exists exactly when

$$\max(d_0, d_1) \leq k, \qquad d_0 + d_1 \leq k + \alpha(H).$$

The refinement procedure extends this cardinality criterion to unequal vote weights, unequal nonnegative control costs, and allowed/required keys. It returns the selected keys, actual admissions, a legal model execution and Bellman lower-bound potentials. A separate checker replays the execution and verifies every local potential inequality, establishing optimality without rerunning the global optimizer. It can also certify infeasibility; a resource limit is reported as unknown. The split-capacity function is not submodular: we give a three-key family on which immediate-gain-per-cost selection has an unbounded approximation ratio.

The supporting theory makes persistence, enabling necessity, observation coverage and the signature deadline explicit. Lean proves the central graph-reversal equivalence, finite event-order realization, and the soundness of Bellman certificates, including infinity-valued infeasibility. Independent event-permutation oracles test the implementation beyond graph-generated orders. A guarded four-validator model supplies the disagreement witness; separately, targeted tests execute unchanged pinned CometBFT consensus code with native signatures and normal commits to check selected implementation obligations.

Order reversal, strongly connected components, feedback vertex sets and certifying algorithms are established tools. Our contribution is the exact translation between phase-control choices and weighted quorum support, its executable optimality certificates, and the connected proof and protocol evidence. The schema must be supplied and its behavioral assumptions discharged; the procedure does not extract a faithful abstraction from arbitrary code. Native conformance tests do not reproduce the model's mutant disagreement or establish a production-network refinement.

# 2 Executions, observations, and scheduling domains

A protocol is a labelled transition system $P = (S, s_0, M, A, \delta, \mathsf{emit})$. A delivery consumes an existing emitted envelope

$$m = (id, v, \lambda, s, r, payload).$$

Here $v$ is the policy's view key, $\lambda$ its phase label, and $s, r$ the **transport endpoints**. A signed message's author need not be its current transport sender: gossip may relay an

unchanged authenticated vote. The bounded model distinguishes these fields and counts certificate support by signer identity.

For a legal finite execution $\tau$, let $O(\mathrm{last}(\tau))$ record decisions at the designated correct observers. The state predicate Unsafe means that these observers have decided conflicting values. Define

$$\mathsf{UnsafeObs}_G(P,\mathcal{B}) = \{O(\mathrm{last}(\tau)) : \tau \in L_G(P,\mathcal{B}),\ \mathsf{Unsafe}(\mathrm{last}(\tau))\}.$$

Our separation claims compare emptiness of these sets. They do not assert equality of every unsafe observation or preservation of a particular fine trace.

A phase-sensitive policy has bits $A_T(v,\lambda,s,r)$. A coarse policy has bits $A_D(v,s,r)$. Phase erasure opens an edge if any phase was admitted:

$$C_{\mathrm{type}}(A_T)(v,s,r) = \bigvee_{\lambda} A_T(v,\lambda,s,r).$$

Admission is only one part of a scheduler. The domain $\mathcal{B}$ also specifies message generation, event opportunities, ordering, pending messages, and termination. The original fixed-order domain uses a stable admission filter and predetermined delivery points. Later sections admit all linear extensions of an explicitly stated partial order. A directed graph by itself implies neither FIFO delivery nor an obligation to process repair before a given vote.

## 2.1 A standard sufficient preservation condition

The useful positive baseline is a specialized simulation argument [9]. Let a candidate coarse trace retain the actions of a fine trace in order and insert closure-required deliveries. Suppose a preorder $\mathcal{R}$ relates the initial states, each insertion increases the target state in $\mathcal{R}$, retained actions preserve $\mathcal{R}$, and unsafety is upward closed in $\mathcal{R}$.

**Theorem 1 (legal relational preservation).** If the candidate trace $C(\tau)$ is legal in the coarse language, $C(\tau) \in L_{G_D}(P,\mathcal{B})$, these conditions imply $\mathsf{Unsafe}(\mathrm{last}(\tau)) \Rightarrow \mathsf{Unsafe}(\mathrm{last}(C(\tau)))$.

**Proof.** Induct over the merged action sequence. At an insertion, the target's increasing step composes with the current relation by transitivity. At a retained action, forward simulation advances both states. The terminal relation and upward closure transfer unsafety. Target legality supplies membership in the coarse execution language. □

Legality is an independent obligation. Processing repair may disable an action or prevent a message from being emitted, so simply retaining all fine votes after inserting repair can produce an illegal sequence. Our negative results instead rerun guarded opportunities; a disabled opportunity emits no vote. Theorem 1 is supporting machinery, not a new general simulation principle.

# 3 A corrected one-pivot separation criterion

Let $q \geq 1$ be a certificate threshold. Two conflicting certificate families have fixed nonpivot support $q-1$ each. The same pivot $p$ is the only possible additional signer. The first and second votes are issued at opportunities $V_0, V_1$; $C_0, C_1$ collect their certificates. Decisions, signed votes, and the availability of emitted votes at the specified collectors are irreversible within the bound.

The following are **behavioral obligations**:

1. **Enabling necessity.** The first pivotal vote is emitted exactly when the enabling envelope $E$ is admitted. No substitute supplies that missing contribution.
2. **Explicit persistence.** When $E$ is admitted and repair $R$ is withheld, $p$ emits the second pivotal vote at $V_1$. Other fixed events do not independently disable it.
3. **Timely repair.** When $E, R$ are admitted and $R$ is processed before $V_1$, $p$ does not emit the second pivotal vote. No previously emitted second-family signature can substitute for it.
4. **Certificate and observation coverage.** The two designated correct observers decide their respective values exactly when the corresponding quorum is available; the bounded agreement observation is unsafe exactly when both families decide. No unrelated decision opportunity creates an additional disagreement.

The **scheduler obligations** are separate: $E, R$ are emitted, share one endpoint/view key, and have distinct phase labels. Fine admission can realize $(e, r) = (1, 0)$; coarse admission realizes only $(0, 0)$ and $(1, 1)$. The relevant order is

$$E < V_0 < C_0 < R < V_1 < C_1.$$

All other delivery choices and opportunities are fixed, and each rerun obeys the behavioral guards. These assumptions deliberately describe a sufficient schema; quorum arithmetic alone cannot establish a protocol refinement.

**Theorem 2 (behavioral quorum-pivot separation).** Under these behavioral and scheduler obligations,

$$\mathsf{UnsafeObs}_{G_T}(P, \mathcal{B}) \neq \varnothing, \qquad \mathsf{UnsafeObs}_{G_D}(P, \mathcal{B}) = \varnothing.$$

**Proof.** Fine admission supplies the first vote by enabling necessity and the second by explicit persistence. Each augments a base of $q-1$, producing two conflicting correct decisions. With the coarse edge off, the first vote is absent and its certificate is short, regardless of whether $p$ can still vote second. With the edge on, the first certificate forms but timely repair prevents the second pivotal signature. Observation coverage makes these two coarse cases exhaustive for agreement. □

The persistence obligation cannot be inferred from the absence of $R$. For example, a fixed local transition after $C_0$ could disable $p$ even when $R$ is suppressed. The endpoint and quorum conditions would still hold, but the fine witness would disappear. Likewise, the order $R < C_1$ is insufficient if $V_1 < R < C_1$: repair cannot retract the already signed vote. Both errors disappear when behavior and signature timing are stated explicitly.

The familiar $n = 3f + 1$, $q = 2f + 1$ family instantiates the two bases with $q - 1$ fixed signatures. Appendix D gives its arithmetic and a $k$-pivot extension; realizability of their behavior remains a separate obligation.

## 4 Repair frontiers for several independent channels

Let $I = \{1, \ldots, k\}$ be pivotal identities, each with positive integer weight $w_i$. Write $w(X) = \sum_{i \in X} w_i$ and $W = w(I)$. The two certificate families have fixed base weights $b_a$, thresholds $q_a$, and deficits $d_a = \max(0, q_a - b_a)$, for $a \in \{0, 1\}$. Unit weights give $W = k$.

## 4.1 Behavioral and order contracts

An admitted $E_i$ enables exactly one first-family signature from pivot $i$. Once the first family is feasible, pivot $i$ emits its second-family signature at opportunity $V_i$ **if and only if** $R_i$ has not been processed before $V_i$. In particular, closing its edge need not prevent its second vote: it was initially ready. No other state transition changes this test. Admitted repair disables prospective issuance; it cannot withdraw signed output. Both collectors and the observation satisfy the coverage obligation of Section 3.

It suffices to impose this second-vote law on policies capable of forming the first certificate. A policy that misses that certificate cannot be unsafe in this two-family observation domain. This qualification accommodates repairs whose commit evidence depends on available first votes.

A finite strict partial order $\prec$ describes opportunities. Repair and vote slots are distinct event kinds: $R_i \neq V_j$ for all $i, j$. The fixed anchors include

$$E_i \prec V_0 \prec C_0 \prec V_i \prec C_1, \qquad V_0 \prec R_i.$$

Additional relations specify the allowed delivery restrictions. Each policy admits **all** linear extensions of this order, interpreted through the guarded machine. Withheld deliveries and disabled voting opportunities stutter. Their slots retain the stated precedence constraints; suppression does not erase a transitive constraint between other events. Every processed envelope must already be emitted and available.

The $k$ endpoint/view keys are independent. A coarse policy opens a subset $A \subseteq I$ and admits $E_i, R_i$ together exactly for $i \in A$. Fine admission can admit all $E_i$ and suppress all $R_i$. These are explicit independence and order-coverage assumptions. If a real harness admits fewer orders, the model gives a sound coarse-safety overapproximation, but an existential witness needs a realizable order in that harness.

## 4.2 A graph of simultaneous repair obstructions

Define a directed graph $H$, including possible self-loops, by

$$i \longrightarrow j \quad \Longleftrightarrow \quad R_i \prec V_j, \qquad \alpha(H) = \max\{|J| : J \subseteq I,\ H[J] \text{ is acyclic}\}.$$

Use transitive precedence, not just a drawing's cover edges. A self-loop means that a pivot's own repair is forced early.

**Lemma 1 (compatible late repairs).** For $J \subseteq I$, a linear extension can place $V_i < R_i$ for every $i \in J$ exactly when $H[J]$ is acyclic.

This is the standard reversible-pair/alternating-cycle argument from partial-order theory, expressed in repair notation [12]. We include the proof because the mapping to scheduling opportunities is central.

**Proof.** Add the required edges $V_i \to R_i$ to the opportunity order. Such an order exists exactly when the augmented relation is acyclic: a finite acyclic relation has a topological ordering. A cycle $i_1 \to \cdots \to i_t \to i_1$ in $H[J]$ yields an augmented closed walk

$$V_{i_1} \to R_{i_1} \prec V_{i_2} \to R_{i_2} \prec \cdots \prec V_{i_1}.$$

Conversely, any augmented cycle must use an added edge, since the original order is acyclic. Between consecutive added edges $V_i \to R_i$ and $V_j \to R_j$, the original-order segment

gives $R_i \prec V_j$, hence $i \to j$. Compress these segments to obtain a nonempty closed walk in $H[J]$, which contains a cycle. This includes self-loops. □

The graph can have cycles even though the opportunity order does not. Its cycles describe incompatible *requests to delay repair*, not a causal cycle in an actual execution.

The full lemma is machine-checked, including existence of the finite topological order. The Lean construction assigns an injective time using the number of strict predecessors and an enumeration index. A separate theorem accepts cover edges and derives the graph from their transitive closure. Thus neither realizability nor the treatment of suppressed slots is an assumed capacity premise.

## 4.3 Exact capacity and separation

**Theorem 3 (repair-frontier characterization).** Under the contracts above, fine unsafety is feasible exactly when $d_0, d_1 \le W$. Coarse unsafety is feasible exactly when there exist $J \subseteq A \subseteq I$ satisfying

$$H[J] \text{ acyclic}, \qquad w(A) \ge d_0, \qquad w(I \setminus A) + w(J) \ge d_1. \tag{1}$$

For unit weights this is equivalent to

$$\mathsf{UnsafeObs}_{G_D} \neq \varnothing \iff \max(d_0, d_1) \le k \ \wedge \ d_0 + d_1 \le k + \alpha(H). \tag{2}$$

Consequently strict phase/coarse separation holds exactly when

$$\boxed{\max(d_0, d_1) \le k \quad \wedge \quad d_0 + d_1 > k + \alpha(H).} \tag{3}$$

**Proof.** Fine admission can supply all first and second signatures by admitting every enabling envelope and suppressing every repair. No branch can receive more than weight $W$, proving its characterization.

For a coarse unsafe execution, let $A$ be its open keys and let $J$ be those open pivots whose second votes precede their repairs. The first support is exactly $w(A)$. Because the first certificate is feasible, the second-vote contract applies: closed pivots remain ready and open pivots contribute second exactly when in $J$. Thus the second support is $w(I \setminus A) + w(J)$. The execution realizes their joint lateness, so Lemma 1 gives acyclicity and establishes necessity of (1).

Conversely, choose $A, J$ satisfying (1). Lemma 1 gives an order in which every member of $J$ signs before repair. Open precisely $A$. The first threshold is met, so the behavioral contract ensures second signatures from every closed pivot and every member of $J$. Further signatures only increase support. Durable outputs reach both collectors; observation coverage gives an unsafe execution.

For unit weights, adding $d_0 \le |A|$ and $d_1 \le k - |A| + |J|$ proves the combined inequality in (2); each demand is at most $k$. For the converse put $h = \max(0, d_0 + d_1 - k)$. Equation (2) implies $h \le \alpha(H)$ and $h \le d_0 \le k$. Choose an acyclic set of size $h$, possible by taking a subset of a maximum acyclic set, and extend it to an open set $A$ of size $d_0$. Then $k - |A| + h \ge d_1$, so (1) applies. Combining fine feasibility with the negation of (2) gives (3). □

Here $h$ is the number of pivots that must contribute to **both** branches. Coarse reachability is decided by whether the scheduler can keep that many jointly reusable contributors, not by the number of admission bits alone.

# 5 Robustness and the amount of phase control required

## 5.1 Common deadlines and crossed deadlines

If all second signatures share one opportunity $V$, let $F = \{i : R_i \prec V\}$ and $\ell = k - |F|$. Members of $F$ have self-loops; all other pivots can jointly place repair after $V$. Hence $\alpha(H) = \ell$ and (2) becomes $d_0 + d_1 \le k + \ell$, subject to individual feasibility. For the $k$-required family, $d_0 = d_1 = k$: separation survives whenever $\ell < k$. Thus allowing some late repairs need not eliminate separation once more than one double contribution is indispensable.

Individual deadlines behave differently. With two pivots and only the critical constraints

$$R_1 \prec V_2, \qquad R_2 \prec V_1,$$

each repair can individually follow its own vote. They cannot both do so: that would require $V_1 < R_1 < V_2 < R_2 < V_1$. The repair graph is a directed two-cycle and $\alpha(H) = 1$, despite having no self-loop. With $d_0 = d_1 = 2$, coarse admission remains safe and phase-sensitive admission is unsafe. This is a separation over several event orders and two independently controlled channels, rather than two executions of one fixed edge.

## 5.2 Optimal phase-splitting budget

A split key admits its enabling phase while suppressing repair. Splitting leaves the opportunity partial order intact. With a chosen split set $T$, the maximum number of double contributors is

$$|T| + \alpha(H[I \setminus T]).$$

Among unsplit keys, Lemma 1 still applies. A split pivot contributes twice without requiring its repair to be late. Choosing at most $t$ split keys gives the exact best capacity $\min(k, \alpha(H) + t)$: this is an upper bound because any unsplit acyclic set has size at most $\alpha(H)$; it is attained by keeping a maximum acyclic set and splitting vertices outside it.

**Corollary 1 (minimum phase budget).** For unit weights, if fine admission is feasible, the exact minimum number of independently split keys needed for an unsafe witness is

$$\boxed{t_* = \max(0, d_0 + d_1 - k - \alpha(H)).} \tag{4}$$

**Proof.** Substitute the best capacity $\min(k, \alpha(H) + t)$ into (2). Since $d_0, d_1 \le k$, the resulting inequality holds exactly when $t$ covers the deficit in (4). □

Figure 1 in Appendix E illustrates the unit-weight boundary. The statement is existential over the choice of split keys. It does not claim that every set of $t_*$ arbitrary message exceptions suffices. A witness can be constructed by choosing an acyclic reusable set, selecting the necessary split keys and open keys, adding its lateness constraints, and topologically sorting the opportunity graph.

## 5.3 Order relaxation, extra edges, and weights

Removing precedence constraints cannot add an edge to $H$, so it cannot decrease $\alpha(H)$. Thus order relaxation weakens separation monotonically under the same behavioral contract

and deficits. Conversely, additional causal paths may increase the needed phase budget. Independent auxiliary events and extra edges are harmless only if they preserve both relevant state/output behavior and the stated order language. An extra substitute vote changes a deficit and must be included in the calculation.

Initially ready and enabling-dependent pivots are different models. With two forced-early pivots and deficits $1, 1$, coarse admission can open one key for the first vote and close the other for the second. Requiring both pivots to see enabling evidence before their second votes would incorrectly remove that coarse witness. Theorem 3 uses the initially ready behavior present in the four-validator edge-off case.

For a common deadline with integer weights, define $L = I \setminus F$ and $W_L = w(L)$. The exact condition is existence of $K \subseteq F$ such that

$$\max(0, d_0 - W_L) \leq w(K), \qquad w(K) + d_1 \leq W. \tag{5}$$

Open every key in $L$ and delay those repairs; choose $K$ among the forced-early keys. The two supports become $W_L + w(K)$ and $W - w(K)$. Opening all $L$ cannot hurt either branch, which proves necessity as well as sufficiency. Indivisible weights matter: weights $2, 2$, no late repair, and deficits $1, 3$ require an unattainable subset weight 1. A total-weight relaxation would give the wrong answer. This is an abstract weighted certificate result, not a weighted CometBFT refinement.

For arbitrary individual deadlines, deciding whether a coarse unsafe execution exists is NP-complete for explicitly represented finite opportunity orders with unbounded $k$. Appendix A gives the short reduction using a classical graph construction [7]; the contribution is the scheduler correspondence, not a new graph-hardness result.

# 6 Certifying minimum-cost phase refinement

Assign each pivot a control cost $c_i \in \mathbb{N}$. Let $D \subseteq I$ be the keys the adapter can split and $Q \subseteq D$ those that must already be split. The synthesis problem selects $Q \subseteq S \subseteq D$, admissible message phases and a linear extension, minimizes $c(S) = \sum_{i \in S} c_i$, and requires both certificate deficits to be met. Costs express a declared control budget, not measured network latency.

## 6.1 An exact weighted choice problem

**Theorem 4 (weighted refinement characterization).** Under the independent-pivot and opportunity-order contracts, a selected set $S$ admits an unsafe model execution exactly when there is a disjoint partition

$$I = S \dot\cup J \dot\cup X \dot\cup Y$$

satisfying

$$H[J] \text{ acyclic}, \qquad w(S \cup J \cup X) \geq d_0, \qquad w(S \cup J \cup Y) \geq d_1. \tag{6}$$

Consequently minimizing $c(S)$ over these partitions and $Q \subseteq S \subseteq D$ gives the exact minimum control cost.

**Proof.** Normalize each split key to admit enabling and suppress repair; under the stated independent contract this cannot reduce either support. Such keys contribute to both families. For unsplit keys, an open key contributes first; a closed key contributes second. Let $J$ be the open keys whose second signatures precede their repairs, $X$ the other open keys, and $Y$ the closed keys. Lemma 1 gives acyclicity and the two support inequalities. Conversely, split $S$, open $J \cup X$, and close $Y$. Topologically sort the complete opportunity order augmented with $V_i < R_i$ for $i \in J$. These choices provide the guaranteed contributions in (6). A member of $X$ may happen to give an additional second vote; this only increases support. Coverage and irreversible outputs give the required observation. □

For a prescribed split set with unit weights, the available double-contribution capacity is $F(S) = |S| + \alpha(H[I \setminus S])$, as used in Corollary 1. The cardinality formula does not choose inexpensive keys or respect inaccessible ones. At full quotas $d_0 = d_1 = W$, (6) requires $X = Y = \varnothing$; the optimization specializes exactly to a weighted directed feedback vertex set, with allowed and required deletions. This classical special case explains the worst-case hardness, not a claim of a new graph problem [7].

## 6.2 Why immediate benefit is an unreliable guide

Consider $H$ with two directed two-cycles $0 \leftrightarrow 1$ and $0 \leftrightarrow 2$, unit vote weights, full quotas 3, and control costs $(M, 1, 1)$ with $M > 2$. Splitting the central key costs $M$; splitting both leaves costs 2 and is optimal. Moreover,

$$F(\varnothing) = F(\{1\}) = F(\{2\}) = 2, \qquad F(\{1,2\}) = 3.$$

The marginal gain of key 1 is zero initially and one after key 2. Thus $F$ is not submodular. A policy selecting the maximum **immediate** gain divided by cost selects the central key and pays $M$, giving ratio $M/2$. This is an unbounded counterexample to that specified myopic rule, not to every heuristic. It also shows why a key with zero immediate benefit can be part of the cheapest sufficient refinement.

## 6.3 Construction and independent certificates

Compute $H$ using reachability in the full opportunity graph. Vertices with self-loops cannot belong to $J$; treat them as individual blocks, then partition the remaining graph into strongly connected components. Every directed cycle of a candidate $H[J]$ lies within one remaining component. Enumerate local assignments to the four roles in (6), respecting allowed and required keys, and reject cyclic reusable sets. Record each assignment's guaranteed supports $(u, v)$ and cost $c$; supports may be capped at the respective deficits.

Let blocks be $C_0, \ldots, C_{m-1}$ and their local alternatives be $\mathcal{A}_j$. Dynamic programming over residual deficits gives

$$L_j(a,b) = \min_{(u,v,c)\in\mathcal{A}_j} \left[c + L_{j+1}\big((a-u)^+, (b-v)^+\big)\right], \tag{7}$$

with $L_m(0,0) = 0$ and $L_m(a,b) = \infty$ otherwise. Backtracking selects roles, admissions and the requested reversals; topological sorting produces a complete event order. A separate event interpreter checks the actual immutable-output execution and its quorum support.

The portable certificate contains the input, chosen execution and potential table. Its checker independently reconstructs transitive reachability, components and **all** legal local assignments. It checks the lower-bound inequalities

$$L_j(a,b) \le c + L_{j+1}\big((a-u)^+, (b-v)^+\big) \tag{8}$$

for every local transition, together with the terminal conditions. Checking only the optimizer's chosen assignments would not certify minimality. Feasibility is checked by replay; the measured witness cost must equal the certified root bound. An external expected input can be supplied so that a valid certificate for another problem is rejected.

**Theorem 5 (certificate soundness).** If all conditions in (8) and the terminal conditions hold, every feasible refinement represented by (6) costs at least $L_0(d_0, d_1)$. A replayed witness attaining a finite root bound is minimum-cost. An infinite root bound certifies infeasibility.

**Proof.** Follow the local assignments of any feasible refinement through the residual-deficit states. Repeatedly applying (8) bounds the initial potential by the accumulated cost plus the terminal potential, which is zero. An infinite potential cannot reach a finite terminal along a finite-cost path. Matching the lower bound with a feasible witness proves optimality. The completeness of the four-role representation, established in Theorem 4, connects this path argument to the scheduling problem. □

Lean checks this Bellman principle for arbitrary nonnegative-cost transition systems, using extended natural bounds. It does not assume that the potentials were computed optimally. The Python parser, graph extraction and exhaustive local enumeration remain executable proof obligations. This follows the established certifying-algorithm approach [10].

Enumeration is exponential in the largest remaining component, and the potential grid has $(m+1)(d_0+1)(d_1+1)$ cells. The dependence on integer quota values is pseudo-polynomial; verification still enumerates local configurations. Neither compact polynomial certificates for all instances nor a polynomial-time optimizer are claimed. Explicit component, table and time limits return a resource-limit result, never a false infeasibility claim.

# 7 Concrete protocol evidence

## 7.1 Source, mutation, and transport

The new model uses CometBFT v0.39.3 at commit `49b82838fcca442b2445f76605c10160 9ed04130` [3, 4]. It follows the exercised proposal, prevote, precommit, valid-block, timeout, and commit guards. A PoLC is a locally received set of three matching prevotes; a commit certificate is three matching precommits from distinct signers. The model has local vote sets and explicit emitted envelopes, with no synthetic network `POLKA` or `COMMIT` messages.

Take one height, equal weights, $n = 4, q = 3$, Byzantine identity $B$, and protocol-following identities $A, P, C$. Only $P$'s round-0 assignment of its persistent locked value and locked round is omitted in the mutant; its valid-block update and its round-1 lock update execute normally. The baseline omits nothing. The two reported conflicting observers, $A$ and $C$, are unmutated. Calling $P$ protocol-following is relative to the explicitly mutated program, not a claim that it follows the original locking specification.

Fix proposers $B$ in round 0 and $C$ in round 1. Initially every correct replica is unlocked, with no valid block or decision. $B$ signs a block-0 proposal and sends it to $A$, which relays the same signed proposal and block to $P$. Thus changing the controlled $B \to P$ edge does not secretly change proposal delivery. That edge's round-0 envelopes are exactly

$$E = B{:}\mathrm{PREVOTE}(0,0), \qquad R = B{:}\mathrm{PRECOMMIT}(0,0),$$

| Program | Round-0 admission | $A$ | $P$ | $C$ | Unsafe |
|---|---|---|---|---|---|
| Baseline | $E$ only | 0 | – | – | no |
| Baseline | edge off | – | 1 | 1 | no |
| Baseline | edge on | 0 | 0 | – | no |
| Missing lock | $E$ only | 0 | 1 | 1 | yes |
| Missing lock | edge off | – | 1 | 1 | no |
| Missing lock | edge on | 0 | 0 | – | no |

**Table 1** Actual terminal decisions in the six canonical guarded executions. A dash means no decision in this finite prefix, not a liveness failure.

where arguments are $(\text{round}, \text{block})$. Both are emitted before $E$ is attempted; $B$ may sign adversarially. All other peer/round admission outcomes are fixed and type-uniform. The complete trace audits this property, distinguishes transport sender from signer, and records suppressed or unavailable delivery opportunities.

## 7.2 The phase-sensitive mutant trace

Write $\mathrm{PV}(r, x)$ and $\mathrm{PC}(r, x)$ for height-1 prevotes and precommits; $\perp$ denotes nil. Self-votes below are actual emitted self-envelopes. Each local quorum is formed from messages already received.

The decisive steps are as follows; Appendix C gives the complete nine-stage trace, and the artifact records every individual envelope and transition.

1. $B$'s block-0 proposal reaches $A$, then $P$ by relay. Both emit prevote 0; the missing-proposal timeout makes $C$ prevote nil. The genuine set $\{A, P, B\} : 0$ gives $A, P$ their local PoLCs. $A$ locks 0. The mutant $P$ keeps valid block 0, omits only its lock assignment, and emits precommit 0.
2. The three precommits from $A, P, B$ make $A$ decide 0. With $R$ suppressed, $P, C$ receive mixed precommit quorums $\{A : 0, P : 0, C : \perp\}$. Their native-rule timeout guards permit a transition to round 1 without a block-0 decision.
3. Unlocked $C$ proposes block 1. The unlocked mutant $P$ prevotes 1 despite retaining valid block 0; it is not the proposer. The actual local prevote sets $\{P, C, B\} : 1$ make $P, C$ precommit 1, with $P$'s normal round-1 lock update retained.
4. $C$ receives the three precommits from $P, C, B$ and decides 1. The conflicting observers $A, C$ contain no injected lock defect. The unmodified baseline retains $P$'s round-0 lock and cannot obtain this second quorum in the fixed schedule.

No unconditional jump between rounds is used. The mixed nil/block precommit set is necessary to justify timeout-driven advancement without giving $P$ the block-0 commit. Nor is valid-block state silently erased by the mutation. The artifact expands these nine groups into every signature, envelope emission, reception, state change, and local certificate.

With the fine baseline, $P$ retains its lock and prevotes 0 in round 1; the second value-1 quorum is absent. With the coarse edge off, $P$ lacks the PoLC-completing $E$, receives a mixed prevote quorum, and precommits nil. $A$ therefore misses its first certificate; $P, C$ may still decide 1. With the edge on in the fixed order, $R$ completes $P$'s actual round-0 precommit quorum. It commits/advances before issuing the required round-1 vote. These are different generated executions, not a table filled with assumed pivot contributions.

## 7.3 Repair before the decision can still fail

Keep the coarse edge on and move $R$. If it arrives after $P$'s round-1 prevote but before its precommit opportunity, the mutant remains safe: that prevote alone cannot supply the third value-1 precommit. If it arrives after $P$ has issued $\mathrm{PC}(1,1)$ but before $C$ decides, the observed decisions are

$$A = 0, \qquad P = 0, \qquad C = 1.$$

The repair commits $P$ to 0 and stops further signing at the height. Its already emitted round-1 votes still reach $C$. When $P$ lacks the block matching its old-round certificate, it explicitly obtains block 0 from $A$ before finalizing; the model does not treat a vote as block data.

The direct version permits a round-1 $B \to P$ prevote to overtake the buffered round-0 repair on that connection. We report this non-FIFO assumption. A separate gossip control obtains the same conflict with FIFO-compatible admitted vote-envelope order: $B$ sends its round-1 prevote to $C$, which relays that unchanged signed vote to $P$; the direct $B \to P$ copy follows $R$. This changes the fixed gossip skeleton and is reported separately. It demonstrates a stronger timing counterexample without silently attributing arbitrary packet reordering to a FIFO transport. Neither control is an execution of the production P2P reactor.

## 7.4 Targeted tests of unchanged implementation code

We separately execute the actual consensus package of the pinned CometBFT revision using Go 1.25.0. Four ordinary-operation scenarios instantiate four unchanged native validators each. Proposals, block parts and votes are taken from their real internal queues; timeout dispatch consumes requests actually scheduled by the implementation. The real proposer-priority algorithm chooses successive proposers. Native vote validation, block validation, the in-memory block store and local ABCI application execute.

The scenarios cover normal commit with missing-block catch-up and emitted-vote immutability, mixed-quorum timeout guards, persistent locking across a legal round change, and proposal reuse of a retained valid block. They pass 14 checks over 32 native Ed25519 votes and 132 recorded events. Three further normal executions preserve the semantic summaries; one race-detector execution also passes. A separate Python/OpenSSL audit verifies all 32 signatures and independently parses their canonical protobuf metadata.

The runner verifies that tracked upstream files are unchanged before and after adding and removing the test overlay. These are targeted checks of implementation obligations, not a native reproduction of the mutant disagreement. Dispatch is synchronous; the reactor, network, WAL loop and persistent validator signer are outside this experiment. Keys and storage use upstream test fixtures. Native timestamps remain untouched, so raw signatures and logs may vary while semantic results remain stable.

# 8 Validation and assurance

The synthesis oracle obtains support pairs directly from raw event permutations and admission choices, without calling graph extraction, SCC routines, the optimizer's event interpreter, or a capacity formula. On all 530 directed graphs with loops for $k \leq 3$, it examines 50,363 legal orders and 1,357,209 executions. It checks 103,460 prescribed-split boundaries and compares 35,223 weighted optimum/infeasibility certificates, including all 219 partial orders

| Synthetic domain | $k$ | $(d_0, d_1)$ | Largest block | Optimum cost |
|---|---|---|---|---|
| Crossed pairs | 48 | $(48, 48)$ | 2 | 75 |
| Two-cycle motifs | 30 | $(30, 30)$ | 3 | 20 |
| Weighted common deadline | 48 | $(72, 77)$ | 1 | 35 |
| Weighted cycles | 24 | $(74, 73)$ | 4 | 5 |
| Dense component | 10 | $(10, 10)$ | 10 | 20 |
| Dense component | 12 | $(12, 12)$ | 12 | unknown |
| Single directed cycle | 24 | $(24, 24)$ | 24 | unknown |

**Table 2** Selected synthesis cases. All finite optima have saved, independently checked certificates. Unknown cases exceed the configured component limit 10; they are not infeasible results or observed timeouts. Costs and graph families are synthetic inputs.

on four event slots and restricted-key cases. Every certificate passes the separate checker; no optimum mismatch occurs.

A second implementation tests 48 arbitrary six-event DAGs, including repair–repair, repair–vote, vote–repair and vote–vote dependencies. Its raw-order oracle examines 4,647 legal orders and 72,117 executions, then compares 3,253 additional optima and certificates with zero mismatches. These finite suites challenge the input-to-graph and support encodings; their execution counts are not numbers of independent protocols.

The structural study contains 15 instances: 13 have certified optima, up to 48 pivots, and two stop at an explicit component-size cutoff. Table 2 shows representative results. Ten copies of the two-cycle motif give optimum 20 while the specified immediate-gain rule pays 1000. The weighted-cycle case has total weight 75: tightening quotas from $(60, 68)$ to $(74, 73)$ changes its optimum from 0 to 5. These are reproducible structural tests, not a runtime contest against specialized feedback-vertex-set solvers. Timings are recorded separately from deterministic results.

The retained frontier evaluator independently executes guarded events and checks the unit boundary and budget on 530 graphs, all 219 four-event orders, 44 common-deadline domains and 258 weighted domains. The last two sweeps use declared support-equivalent or extremal order representatives. Complete case counts, raw support observations, input fixtures and all 13 scale certificates are included in the artifact.

The concrete trace has independent post-execution audits for prior emission, once-only envelope delivery, authentication, distinct actual signers, local certificate timing, signing guards, and quorum-enabled timeouts. Negative controls reject fabricated envelopes, altered authenticators, duplicate votes or deliveries, insufficient PoLC evidence, and unjustified timeout transitions. The canonical mutant and the late-repair mutant expose disagreement at $A, C$, which contain no injected lock defect.

## 8.1 What Lean checks

The primary project is pinned to Lean 4.33.0. The revision adds a theorem over an **arbitrary behavioral contract** for Theorem 2, explicitly assuming positive persistence and the observation contract rather than defining the desired safety classification. It also checks the arithmetic capacity equivalence, its strict-separation form, the minimum split budget under its capacity contract, the weighted admission interval, and a small output machine in which later repair preserves an earlier signature. The build includes an axiom audit and independent rechecking of the resulting kernel objects.

The new graph module proves reversal equivalence, finite acyclicity versus an injective

schedule, and the graph-to-event-order bridge, including input given as cover edges. It does not assume $\alpha(H)$ or a capacity formula. A second module proves Theorem 5's lower-bound principle, minimum-cost witness rule and infeasibility rule over arbitrary costed paths with extended-natural potentials. The integrated audit covers 27 named declarations. The weighted four-role correspondence, completeness of the Python checker and protocol-specific behavioral contracts remain paper proofs or executable obligations; the concrete traces are not a Lean refinement of CometBFT.

Earlier Lean, SMT, certificate-kernel, Python, and Go checks remain available. In particular, agreement of 384 Python/Go rows for $f = 1, \ldots, 64$ checks the previous Boolean model's encoding. The old theorem named `bounded_tendermint_refines_quorum_pivot` establishes matching terminal classifications, not a transition refinement. Increasing $f$ in that family is not independent validation of many protocols. The revised claim ledger records this distinction.

## 9 Related work and the novelty boundary

**Communication closure.** Drăgoi et al. compare asynchronous executions with lossy synchronous executions under implementation conditions and develop testing based on communication closure [5]. Their communication rounds distinguish send/receive/update steps, including different protocol phases. Our operator additionally couples admission across such steps within one view key. The counterexample therefore does not refute their normalization theorem or the effectiveness of their tester; it identifies an obligation for a further aggregation.

**Controlled and adaptive testing.** Twins already describes certificate arrivals advancing a stale twin and supports message-type filtering [2]. ByzzFuzz, Netrix and ByzzBench supply randomized fault testing, programmable implementation testing and benchmark infrastructure [13, 6, 1]. FEST adapts schedule generation using behavioral and scenario feedback while retaining its underlying scheduler constraints [8]. Our procedure solves a different, conditional design problem: selecting the cheapest phase distinctions for a supplied finite quorum/order contract and certifying that optimum. We do not compare bug-finding speed or rank these frameworks.

**Preservation and partial orders.** Simulation with internal actions is established machinery [9]. The inconsistent-labelling literature shows why action-oriented reduction conditions must be connected carefully to state observations [11]; its operator differs from phase admission closure. Reversibility of incomparable pairs is characterized by absence of alternating cycles in classical partial-order theory [12]. Lemma 1 uses that argument, and the induced-acyclic optimization and symmetric-graph hardness construction are classical [7]. The result developed here combines this order characterization with initial readiness, irreversible vote issuance, and certificate deficits, yielding the exact scheduler phase budget, weighted control synthesis and a guarded BFT-rule witness. The independent-witness/lower-bound design follows certifying algorithms; its value here is the complete input-to-schedule certificate for the declared schema, not a new general certification principle [10].

The literature audit supports this specific positioning, not an assertion that no equivalent result exists in all prior literature. Replay congruence and the earlier seven-flag diagnostic are retained as supplementary results in Appendix B; they do not carry the main novelty claim.

## 10 Limitations and reproducibility

The strongest mathematical statements are conditional on the independent-pivot and complete opportunity-order contracts. New edges that supply substitute support, policies that change message generation beyond these contracts, changing validator sets, recovery, and interdependent locking behavior require a new conformance argument. The weighted result concerns indivisible certificate weights with fixed membership. It is not a staking or dynamic-quorum theorem.

The artifact contains exact v0.39.3 source files, verified Git blob identifiers and SHA-256 hashes; the older snapshot is separately retained as v0.40.0. The local mutant model uses test authentication, a supplied proposer prefix and aggregate block deliveries. Native conformance exercises the real signing, proposer and block routines separately, but supplies no formal refinement of that mutant trace. The four-role theorem and certificate principle do not automatically establish the behavioral contract of a richer client.

The public working artifact is available at `https://github.com/galetaa/twins-scheduler`. The command `./reproduce_revision.sh` runs the model tests, frontier and synthesis studies, both synthesis oracles, native conformance, independent signature audit, repeated/race executions, Lean build and axiom audit, and LIPIcs build. A mandatory failure stops the run. Deterministic results are compared with committed references; native raw timestamps and signatures are revalidated rather than byte-compared. Source pins, complete certificates, claim mappings and tool-version logs accompany the manuscript. Dependencies require network downloads on an empty cache.

## 11 Conclusion

Phase control can be synthesized as a minimum-cost design choice once the relevant quorum and opportunity contracts are explicit. The repair graph identifies incompatible reusable contributors; weighted role assignments connect them to certificate deficits. The resulting algorithm returns a legal model execution and independently checked optimality evidence. Non-submodular control value explains why immediate-benefit selection can miss inexpensive combinations. Lean closes the central graph/order and certificate-principle proofs, while separate native conformance tests strengthen selected implementation assumptions. The remaining obligation is to establish these contracts for each richer protocol instance and adapter.

### 11.1 Acknowledgments and AI-use disclosure

An OpenAI language model assisted with research organization, literature discovery, proof exploration, manuscript drafting, code generation, and consistency checking. The authors retain responsibility for the correctness, originality, citations, and experimental claims. The model is not an author.

## References


1. Jo ao Miguel Louro Neto and Burcu Kulahcioglu Ozkan. A benchmark framework for byzantine fault tolerance testing algorithms. In *6th International Workshop on Formal Methods for Blockchains (FMBC 2025)*, volume 129 of *Open Access Series in Informatics (OASIcs)*, pages 13:1–13:11, 2025. doi:10.4230/OASIcs.FMBC.2025.13.
2. Shehar Bano, Alberto Sonnino, Andrey Chursin, Dmitri Perelman, Zekun Li, Avery Ching, and Dahlia Malkhi. Twins: BFT systems made robust. In *25th International Conference on Principles of Distributed Systems (OPODIS 2021)*, volume 217 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 7:1–7:29, 2022. doi:10.4230/LIPIcs.OPODIS.2021.7.
3. CometBFT Project. Cometbft consensus specification. Release v0.39.3, commit prefix 49b8283, 2026. Engineering specification pinned for the bounded instantiation. URL: https://github.com/cometbft/cometbft/blob/v0.39.3/spec/consensus/consensus.md.
4. CometBFT Project. CometBFT consensus state implementation, 2026. Release v0.39.3. Exact source snapshot and integrity manifest included in the artifact. URL: https://github.com/cometbft/cometbft/blob/49b82838fcca442b2445f76605c101609ed04130/consensus/state.go.
5. Cezara Drăgoi, Constantin Enea, Burcu Kulahcioglu Ozkan, Rupak Majumdar, and Filip Nikšić. Testing consensus implementations using communication closure. *Proceedings of the ACM on Programming Languages*, 4(OOPSLA):210:1–210:29, 2020. doi:10.1145/3428278.
6. Cezara Drăgoi, Srinidhi Nagendra, and Mandayam Srivas. A domain specific language for testing distributed protocol implementations. In *Networked Systems (NETYS 2024)*, Lecture Notes in Computer Science, pages 100–117, 2024. doi:10.1007/978-3-031-67321-4_6.
7. Richard M. Karp. Reducibility among combinatorial problems. In Raymond E. Miller and James W. Thatcher, editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972. doi:10.1007/978-1-4684-2001-2_9.
8. Ao Li, Ankush Desai, and Rohan Padhye. Feedback-guided adaptive testing of distributed systems designs. In *23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*, pages 701–719. USENIX Association, 2026. URL: https://www.usenix.org/conference/nsdi26/presentation/li.
9. Nancy A. Lynch and Frits W. Vaandrager. Forward and backward simulations: I. untimed systems. *Information and Computation*, 121(2):214–233, 1995. doi:10.1006/inco.1995.1134.
10. Ross M. McConnell, Kurt Mehlhorn, Stefan Näher, and Pascal Schweitzer. Certifying algorithms. *Computer Science Review*, 5(2):119–161, 2011. doi:10.1016/j.cosrev.2010.09.009.
11. Thomas Neele, Antti Valmari, and Tim A. C. Willemse. A detailed account of the inconsistent labelling problem of stutter-preserving partial-order reduction. *Logical Methods in Computer Science*, 17(3):8:1–8:27, 2021. doi:10.46298/LMCS-17(3:8)2021.
12. Noah Streib and William T. Trotter. Dimension and height for posets with planar cover graphs. *European Journal of Combinatorics*, 35:474–489, 2014. doi:10.1016/j.ejc.2013.06.017.
13. Levin N. Winter, Florena Buşe, Daan de Graaf, Klaus von Gleissenthall, and Burcu Kulahcioglu Ozkan. Randomized testing of byzantine fault tolerant algorithms. *Proceedings of the ACM on Programming Languages*, 7(OOPSLA1):757–788, 2023. doi:10.1145/3586053.

# A Complexity of coarse witness reachability

In the unweighted independent-pivot schema, the decision problem of whether a coarse unsafe execution exists is NP-complete when the number of pivots is unbounded and the finite opportunity order is represented explicitly by its events and precedence or cover edges. A certificate consists of an open set, a reusable set and a topological event order; checking admission, ordering and quorum support takes polynomial time.

For hardness, take an undirected graph on $k$ vertices and an independent-set target $t$. Introduce repairs $R_i$ and second-vote opportunities $V_i$. For every undirected edge $\{i, j\}$ impose $R_i \prec V_j$ and $R_j \prec V_i$. These constraints are acyclic: they point from the repair layer to the vote layer. The fixed prefix and certificate collectors can be added before and after those layers. The resulting repair graph is the symmetric orientation of the original graph. Its induced set is acyclic exactly when it is independent, since each undirected edge creates a directed two-cycle.

Set $d_0 = k, d_1 = t$. Theorem 3 gives coarse unsafe reachability exactly when $\alpha(H) \geq t$, which is the independent-set question. This uses the classical relation between node cover, feedback node sets, and symmetrized graphs [7]. It is a complexity statement about the declared opportunity schema. It neither establishes a complexity bound for the fixed four-validator trace nor claims that every constructed order is realizable by a production consensus implementation.

# B Supporting replay and diagnostic results

## B.1 Replay erasure with outputs

Let $\pi : S \to S_{\mathrm{rel}}$ project relevant state, including processed message keys. For every retained message $m$, require **state congruence** and **output congruence**:

$$\pi(s) = \pi(t) \Rightarrow \pi(\delta(s, m)) = \pi(\delta(t, m)), \qquad \pi(s) = \pi(t) \Rightarrow \mathsf{out}(s, m) = \mathsf{out}(t, m).$$

For an exact marked duplicate whose key was already processed, require projected stutter $\pi(\delta(s, m)) = \pi(s)$ and output silence $\mathsf{out}(s, m) = \epsilon$.

**Supporting proposition (replay erasure).** Deleting all marked duplicates preserves the terminal relevant projection and the complete relevant output trace under these conditions.

**Proof.** Maintain equality of the two current projections and accumulated outputs. A duplicate changes neither on the unerased side. At a retained message, the two congruences preserve the invariant. Induction gives the terminal claim. □

Projected silence alone is insufficient: in a trace $X, X, Y$, the duplicate can change a hidden bit that the later $Y$ copies into relevant state or output. The artifact retains these negative controls. This proposition is standard congruence reasoning; it supports the distinction between internal state and persistent outputs without constituting the revised paper's main contribution.

## B.2 The earlier seven-flag diagnostic

For Boolean flags $S_1, S_2, R_q, D, T, E_d, X$, the earlier operational abstraction uses

$$U(e, r) = (\neg S_1 \vee e) \wedge [\neg S_2 \vee ((\neg R_q \vee e) \wedge \neg(r \wedge D \wedge T))].$$

Here the shortage flags describe the two bases; $R_q$ makes second participation enabling-dependent; $D$ gives repair efficacy; $T$ means repair precedes the **second-vote opportunity**; $E_d$ couples the endpoint key; and $X$ permits phase distinction. Direct substitution yields

$$X \wedge E_d \wedge U(1,0) \wedge \neg U(0,0) \wedge \neg U(1,1) \iff S_2 \wedge D \wedge T \wedge E_d \wedge X \wedge (S_1 \vee R_q).$$

Exactly three of the 128 assignments satisfy this expression. This is a diagnostic of that Boolean machine. The old atomic opportunity interpretation combined vote and certificate formation; it must not be reused to infer that repair merely before a later collector suffices. The revised frontier model separates these events and makes initially ready pivots the primary multi-pivot case.

## C Detailed guarded four-validator model trace

1. $A$ receives $B$'s block-0 proposal and signs $\mathrm{PV}(0,0)$. $A$ relays the proposal to $P$, which signs the same prevote. $C$ receives no proposal; its proposal timeout produces $\mathrm{PV}(0,\perp)$.
2. $B$ emits its round-0 prevote and precommit. Deliver $P$'s and $B$'s prevotes to $A$. The set $\{A,P,B\}:0$ makes $A$ persist valid/locked block 0@0 and sign $\mathrm{PC}(0,0)$.
3. Deliver $A$'s prevote and $E$ to $P$. Its same three-signer PoLC sets valid block 0@0. The mutation omits only its lock assignment, and $P$ signs $\mathrm{PC}(0,0)$ while remaining unlocked.
4. Deliver $C$'s nil prevote to $P$, and the $A,P$ block-0 prevotes to $C$. At $C$, the mixed set $\{A:0,P:0,C:\perp\}$ enables its prevote timeout without creating a block-0 PoLC. On expiry, $C$ signs $\mathrm{PC}(0,\perp)$.
5. Deliver $A$'s precommit to $P,C$, and $P$'s precommit to $A,C$. Deliver $B$'s precommit to $A$. Its local set $\{A,P,B\}:0$ reaches three and $A$ decides 0.
6. Deliver $C$'s nil precommit to $P$. Both $P$ and $C$ now have the mixed set $\{A:0,P:0,C:\perp\}$. It enables a precommit timeout but does not commit block 0. The fine policy suppresses $R$. Expiry advances $P,C$ lawfully to round 1.
7. $P$ retains valid block 0@0 but no lock; $C$ has neither. Consequently the unchanged proposer $C$ can propose new block 1 with valid round $-1$. $C$ and the unlocked $P$ each sign $\mathrm{PV}(1,1)$. Making $P$ the proposer instead would require accounting for its cached valid block 0.
8. $B$ emits round-1 prevote and precommit for 1. Deliver the $C,B$ prevotes to $P$, producing PoLC $\{P,C,B\}:1$. $P$ normally updates valid/locked block to 1@1 and signs $\mathrm{PC}(1,1)$. Deliver the $P,B$ prevotes to $C$; it obtains the same PoLC and precommits 1.
9. Deliver the $P,B$ precommits to $C$. Together with $C$'s self-precommit, they produce decision 1. Delivering the $C,B$ precommits to $P$ also lets it decide 1. The unmutated observers have $A=0, C=1$.

## D Parametric certificate families

For every $f \geq 1$, take $n = 3f+1$, $q = 2f+1$, Byzantine set $B$ of size $f$, correct exclusive sets $A,C$ of size $f$, and pivot $p$. Then

$$Q_0 = B \cup A \cup \{p\}, \qquad Q_1 = B \cup C \cup \{p\}$$

have size $q$ and each nonpivot base has size $q-1$. Under the behavioral contract, Theorem 2 applies. This arithmetic does not assert that every locking protocol realizes the contract.

More generally, for $1 \le k \le f$, use $k$ pivots, exclusive sets $A, C$ of size $f+1-k$, Byzantine set $B$ of size $f$, and $k-1$ unused correct identities. The total is still $3f+1$. Both quorums contain $B$, their exclusive set, and all $k$ pivots; each base has size $q-k$. Distinct correct observers exist in nonempty $A, C$. Theorem 3 characterizes precisely which opportunity orders can preserve these $k$ required double contributions. The extension is a certificate schema; the concrete protocol trace in Section 7 establishes the single-pivot model case.

## E Unit-weight budget illustration

The heat maps instantiate Corollary 1 at three reusable capacities. They illustrate the cardinality special case; unequal control costs require the optimization in Theorem 4.

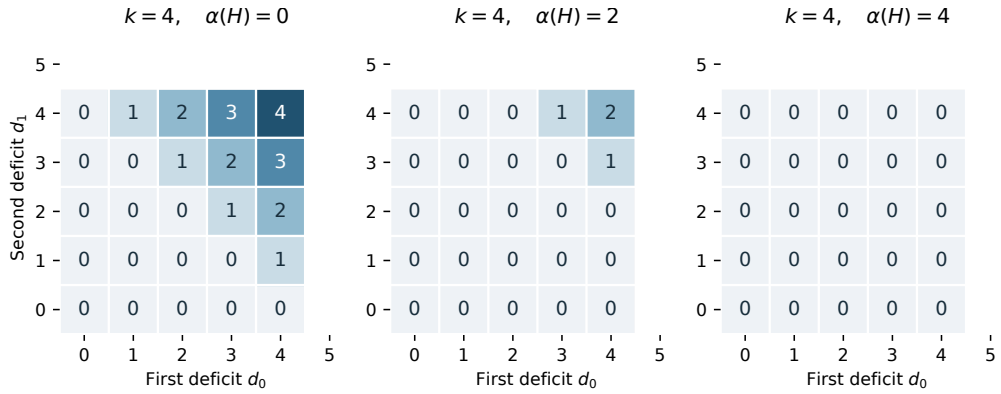



**Figure 1** Exact phase-control budget in the independent-pivot schema. Each cell is the minimum number of phase-split keys required to expose an unsafe witness. Blank cells exceed the available pivot count. Panels differ only in the capacity of jointly late repair, not in quorum size.